ATC'26 · HONG KONG · NOVEMBER 2026

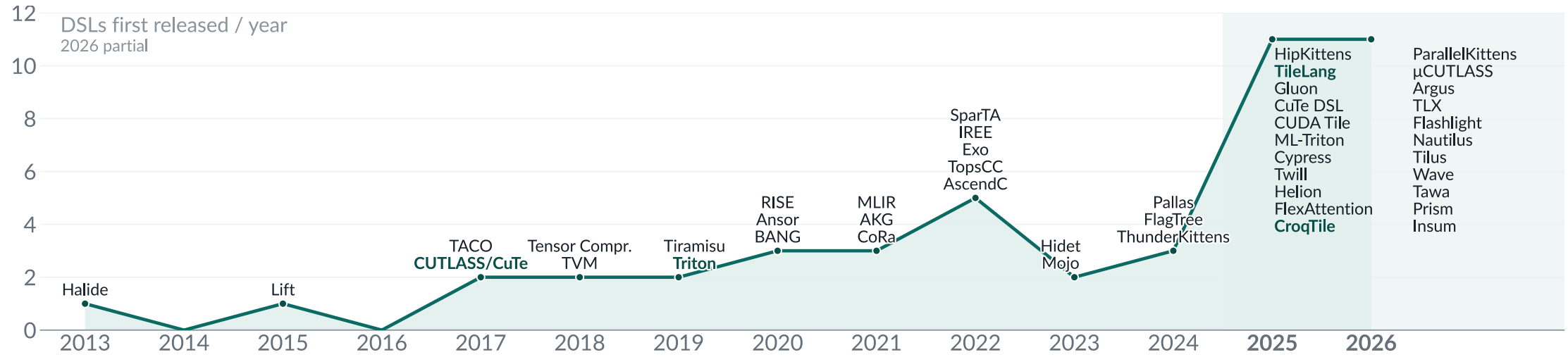
A Novel AI-Native GPU Kernel Programming Language for Autonomous Kernel Tuning

Heng Shi · Xiaofeng Guan · Sijie Wu · Enming Fan · Chenpeng Wu
Qiqi Gu · Jiahang Xie · Guoqing Bao · Jianguo Yao

Shanghai Enflame Intelligence & Shanghai Jiao Tong University

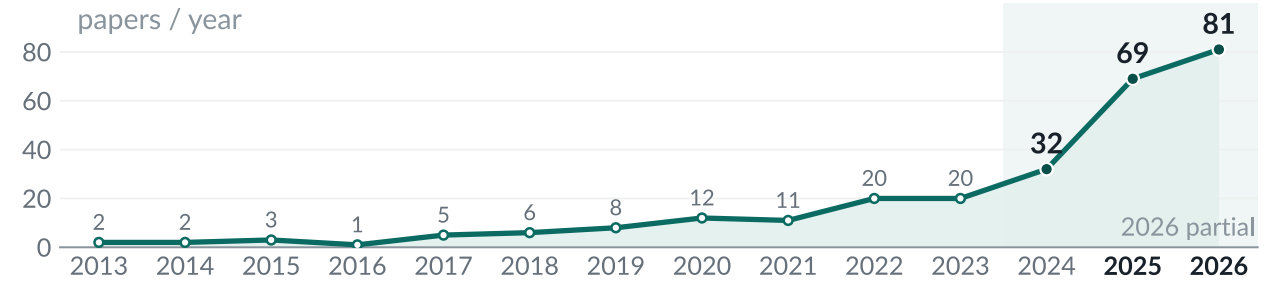
Operator work keeps growing

Model families converge. ***Kernel variants multiply.***



22/46 **DSLS IN 2025–26**

- **More programming substrates**
  46 DSLs in the survey corpus.
- **More operator work**
  276 optimization papers across 2013–26.



Why kernels multiply

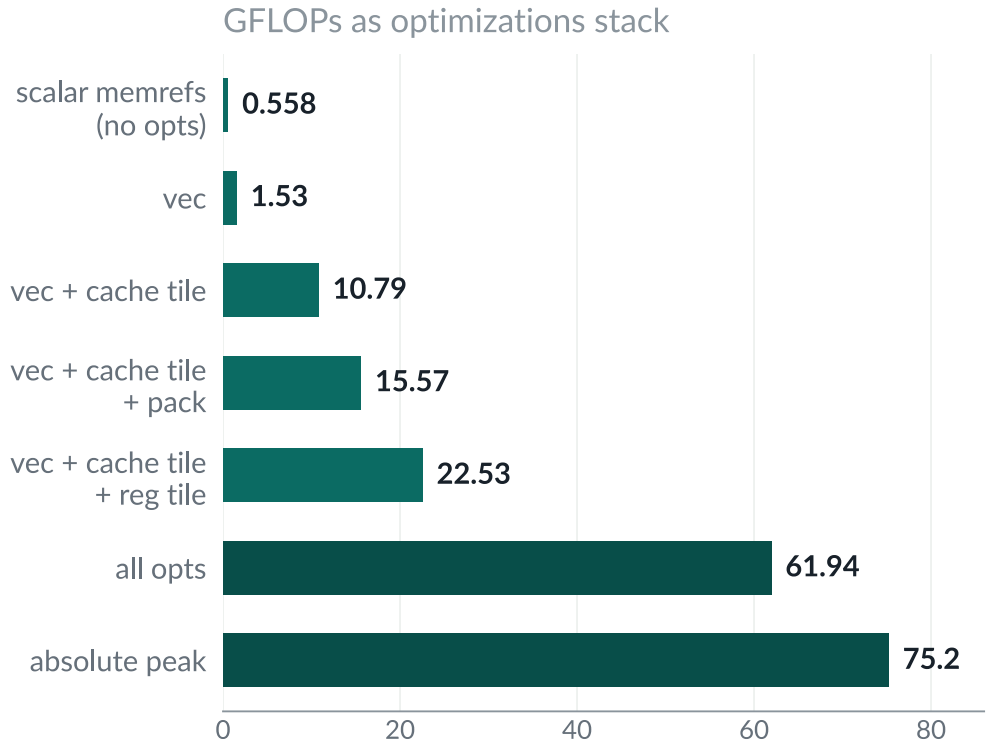
1	**Operator variants**	MHA · MQA · GQA · MLA · sliding-window · block-sparse Attention keeps branching
2	**Hardware diversity**	NVIDIA · AMD MI300X · Ascend · TPU · Trainium One kernel, many targets
3	**New GPU instructions**	Hopper: WGMMA + TMA → Blackwell: tcgen05 + TMEM New instructions, new dataflows
4	**Operator fusion**	Conv + BN + ReLU → fusion changes the kernel boundary Eliminate intermediate traffic

The maintenance burden multiplies. Operator variants × hardware targets × optimization choices.

Kernel optimizations interact

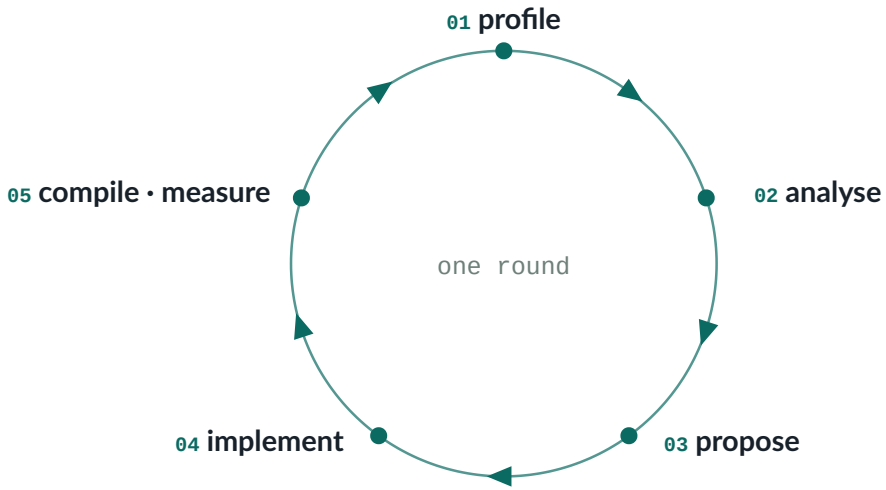
COUPLED PERFORMANCE DECISIONS

- **Locality**
  Tiling · data reuse · L2 bypass / cache policy
- **Data movement**
  Swizzling · vectorization · async copies
- **Execution overlap**
  Multibuffering · warp specialization · scheduling
- **Compute & cooperation**
  Unrolling · approximation · sparsity · fusion
  MMA layouts · low-precision MMA · clusters / DSMEM



A larger tile improves reuse… …but consumes more shared memory, limiting stages and overlap.

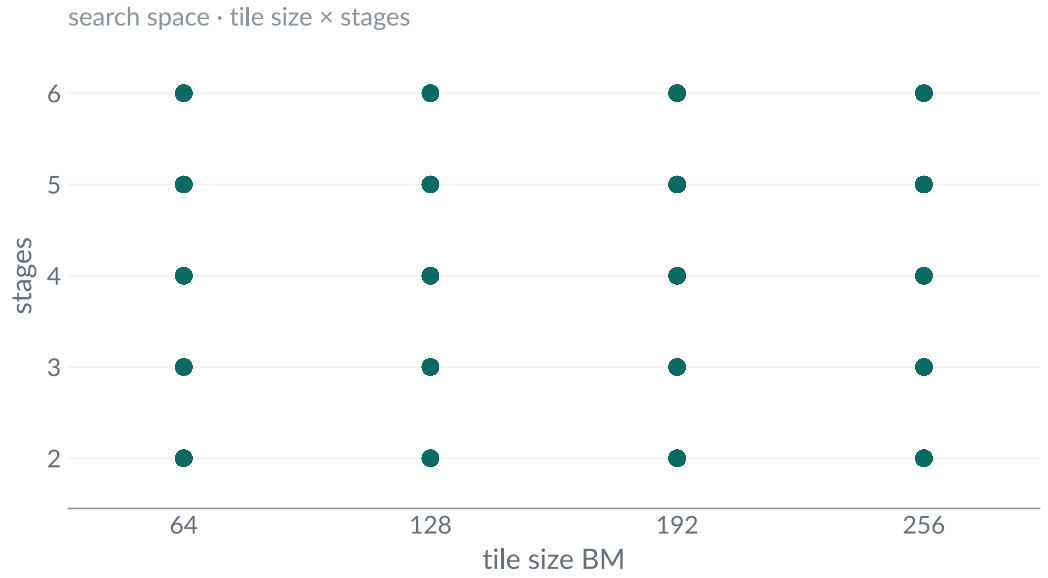
Bottleneck-driven tuning



1. Profile
2. Identify bottleneck
3. Choose structural edit
4. Implement
5. Compile · measure · keep / reject

- **Example: tensor cores waiting for data**
  Introduce a producer–consumer pipeline.
- **Expert effort in every round**
  Interpret counters, redesign the kernel, validate the edit.

Template search



`Tile size · stages · warps · swizzle`

- **Fixed skeleton**
  Choose the implementation structure up front.
- **Parameter search**
  Enumerate tile sizes, stages, warps and swizzles.
- **Filter, then benchmark**
  Reject resource violations; measure valid candidates.

Boundary: a new producer–consumer structure must first be exposed in the template.

Compiler transformations

data tiling / blocking	→ **`Loop tiling`**
data stationary	→ **`Cache staging`**
multibuffering & prefetch	→ **`Software pipelining`**
loop unroll	→ **`Unrolling`**
vectorisation	→ **`Vectorization`**
loop fusion	→ **`Fusion`**
instruction scheduling	→ **`Instruction scheduling`**

`MLIR · TVM · IREE`

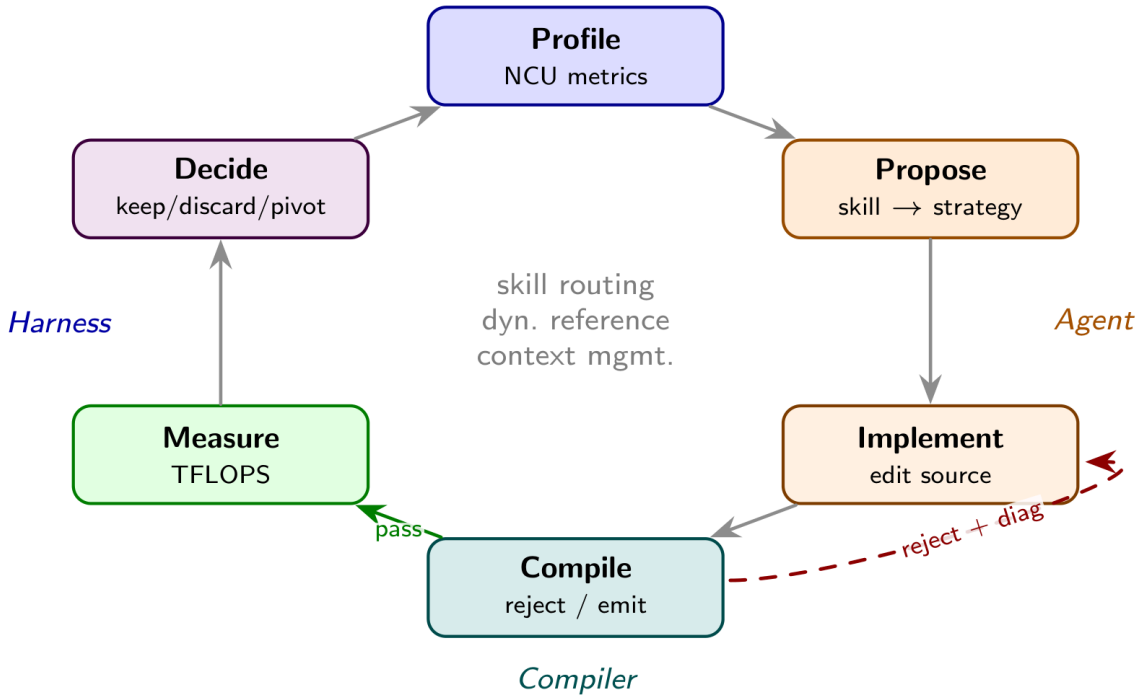
- **Schedule DSL**
  Express transformations in Halide / TVM TE.
- **Compiler passes**
  Encode legal rewrites over loops and dataflow.

New hardware, new structure: someone still needs to implement the corresponding transformations.

Agents rewrite the kernel

PROFILE → EDIT → EVALUATE

Full source edits · structural + parametric



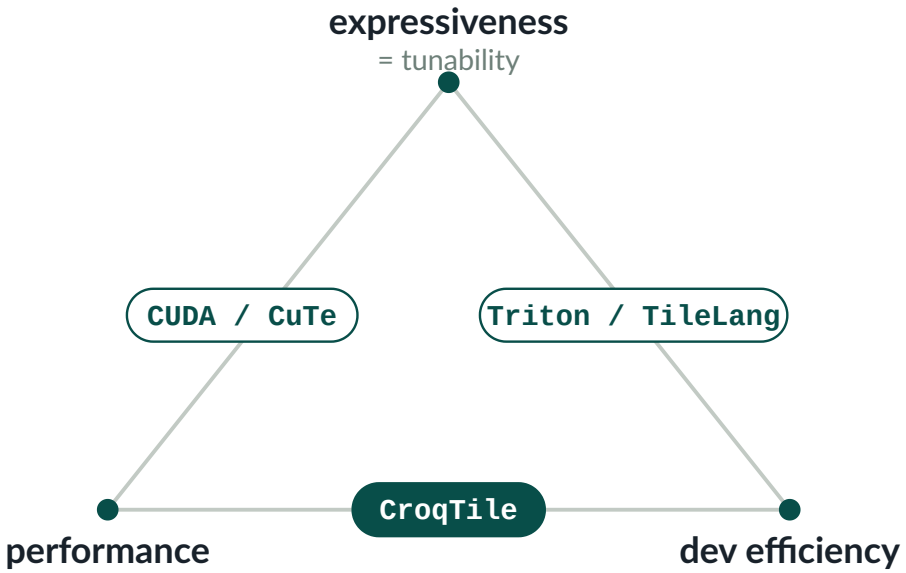
WORKFLOW	Planning · search · refinement
EVALS	Correctness · speed · generalization
TOOLING	Syntax–context · compiler–harness co-design
KNOWLEDGE	Skills · references · history

Design for agent failures

FAILURE	REQUIREMENT
Context pressure Source + profiler output + debugging history	**Compact source** Compact programs leave room for prior attempts and measurements.
Coupled edits TMA: descriptor, swizzle, copy, synchronization	**Independent primitives** Give each performance decision an explicit, local edit site.
Opaque failures GPU hang or an error without a source location	**Early, actionable feedback** Reject early; report the constraint, edit site and repair.

CroqTile: two co-design axes

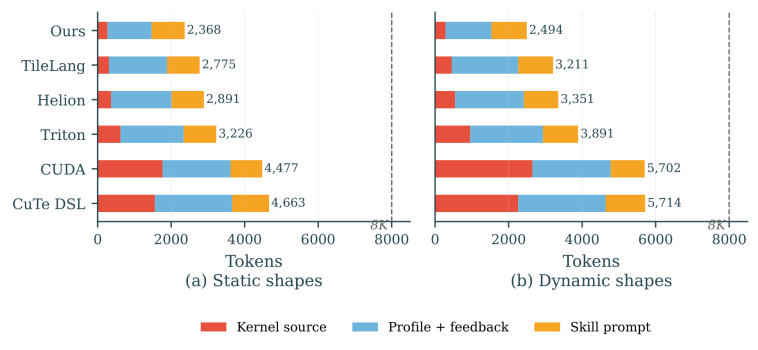
EXPRESSIVENESS · PERFORMANCE · ITERATION COST

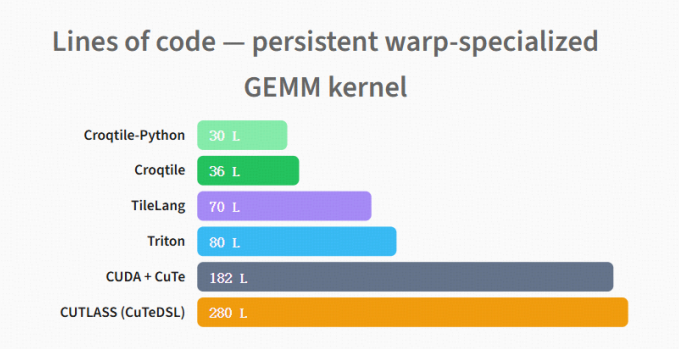


`CUDA / CuTe`	**Explicit control** Long source · coupled edits
`Triton / TileLang`	**Compact tile operations** Some structural controls hidden

- **Syntax–context co-design**
  Compact syntax and fewer coupled edit sites leave room for source, feedback and tuning history.
- **Compiler–harness co-design**
  Symbolic checks and structured diagnostics guide rejection, repair and GPU evaluation.

Compact syntax preserves context





Structure

$kernel \rightarrow$ `__co__ void id (` $param^*$ `) {` $stmt^*$ `}`
$param \rightarrow$ `[`$addrsp$`]` $dtype$ `[`$shape^+$`] id`
$addrsp \rightarrow$ `global` | `shared` | `local`
$stmt \rightarrow$ $decl$ | $parallel$ | $foreach$ | $warpspec$ | dma | mma | $event$ | $sync$ | $assign$ | ...

Parallelism & Iteration

$parallel \rightarrow$ `parallel` $binds$ `:` $space$ `{` $stmt^*$ `}`
$binds \rightarrow$ `id by` $expr$ | `{ id`$^+$ `} by [`$expr^+$`]`
$space \rightarrow$ `block` | `group-`K | `cluster`
$foreach \rightarrow$ `foreach { id`$^+$ `} in [`$expr$`] {` $stmt^*$ `}`
$warpspec \rightarrow$ `inthreads.async (`$expr$`) {` $stmt^*$ `}`

Declarations

$decl \rightarrow$ `[`$addrsp$`]` $dtype$ `[`$shape^+$`] id`
$event \rightarrow$ `shared event id[`N`] (, id[`N`])`*

Data Movement

$dma \rightarrow$ `(tma|dma).copy[.async<`ev`>]` $attr^*$ ref `=>` ref
$attr \rightarrow$ `.swiz<`N`>` | `.zfill` | `.sparse`

Compute

$mma \rightarrow$ `mma.`op `[.`mod`]`* ref^*
$op \rightarrow$ `fill` | `row.row` | `row.col` | `col.col` | `commit` | `store`
$mod \rightarrow$ `sp` | `scale`

Synchronization

$sync \rightarrow$ `wait id[`$expr$`]` | `trigger id[`$expr$`]`

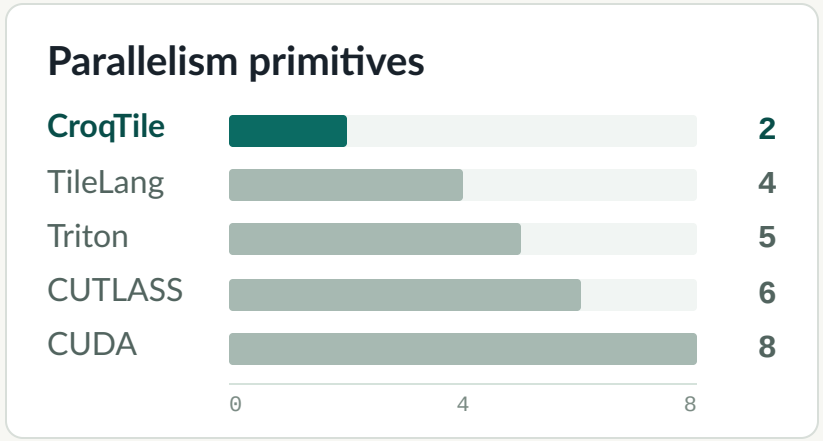
- **36 vs. 182 source lines**

  Same persistent, warp-specialized GEMM; CroqTile vs. CUDA + CuTe.

- **Room for tuning history**

  ≈2.37K vs. ≈4.48K tokens per static-shape attempt, including source, feedback and skills.

parallel: one construct, three levels



- **One binding, three levels**

  `block` · `group-4` · `thread`

- **Explicit producer–consumer roles**

  `inthreads.async` separates TMA producers and MMA consumers.

- **Fewer coupled edit sites**

  Change the parallel binding without reconstructing thread-index calculations.

CroqTile | TileLang | CuTe / CUTLASS | CUDA

```
parallel {bm, bn}
  by [cdiv(M, WARP_M), cdiv(N, WARP_N)] : block {
  parallel p by 1 : group-4 {
    mma.row.row mc, ma, mb;
  }
  parallel {m, n}
    by [WARP_M, 2] : thread { ... }
}

// Producer–consumer warp specialization
parallel p1 by 2 : group-4 {
  inthreads.async (p1 == 0) { /* TMA producer */ }
  inthreads.async (p1 == 1) { /* MMA consumer */ }
}
```

tma: one statement per transfer



- **One statement per transfer**
  Source and destination specify the tile movement.
- **Swizzle as a modifier**
  `.swiz<128>` makes the layout choice explicit at the transfer.
- **Compiler-generated descriptor**
  The agent edits the transfer; the compiler keeps the derived descriptor consistent.

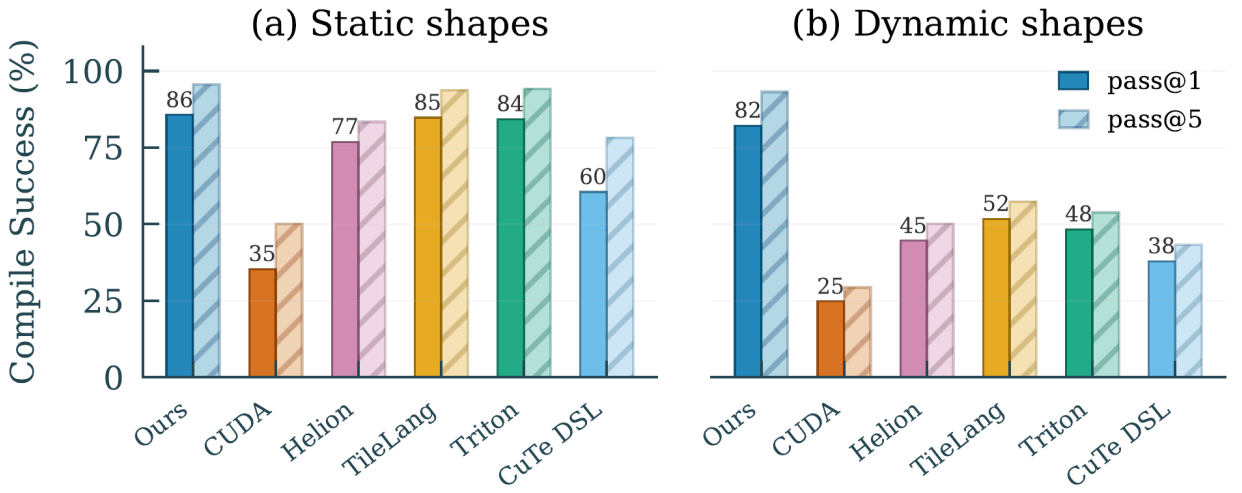
CroqTile | Triton | TileLang | CuTe / CUTLASS | CUDA

```
// LOAD · global → shared
tma.copy.swiz<128>
  lhs.subspan(WARP_M, TILE_K)
     .at(bm, iv_k) => lhs_s;

// STORE · shared → global
tma.copy out_s =>
  output.subspan(WARP_M, WARP_N)
        .at(bm, bn);
```

Compile success across shapes

PASS@1 / PASS@5 · STATIC + DYNAMIC



pass@1: 85.7% static · 82.1% dynamic
Dynamic shapes: Triton 48% in this evaluation.

1 Less coupled syntax

Each optimization touches fewer interdependent code sites.

Fewer details to keep consistent → fewer edit errors.

2 Symbolic inference

The type system and compiler infer shape relationships and check constraints.

Less manual shape bookkeeping, especially for dynamic shapes.

Diagnostics that guide repair

CONSTRAINT · OBSERVED · LOCATION · REPAIR

```
<compile_result status="rejected" config="stg3,128x128" tokens="74">
  <constraint cat="shape-compat">
    DMA dim-0 mismatch. predicate: from.d0 == to.d0
  </constraint>
  <observed>
    from.d0 = (N / cdiv(N, 128))
    to.d0 = 128
    valid: to.d0 must equal from.d0
  </observed>
  <location file="spmm.co" line="58">
    tma.copy.async<full[stg]>.swiz<128>
      rhs.subspan(WN,TK).at(bn,k)
      => rhs_s.subspan(WN,TK).at(stg,0);
  </location>
  <repair priority="high" confidence="certain">
    Do: Append .zfill to the DMA — zero-fills tail tiles
      where N % 128 != 0.
    Why: Source tile < dest when N is not a multiple of 128.
    Alt: Set tile_n = cdiv(N,128)*128 so from.d0 == to.d0.
  </repair>
</compile_result>
```

1 Symbolic inference

Check shape compatibility and resource limits before launch.

2 Runtime assertions

Check constraints that depend on runtime values.

3 Compiler–harness interface

Pass **constraint, observed, location and repair** as structured fields.

4 Repair in the tuning loop

The harness returns the rejection to the agent; the diagnostic identifies the statement and corrective edit.

Prune before GPU execution

SPARSE GEMM · CONFIGURATION PRUNING



- **353 checks across 7 modules**

  Compiler checks cover shapes, resources and synchronization.

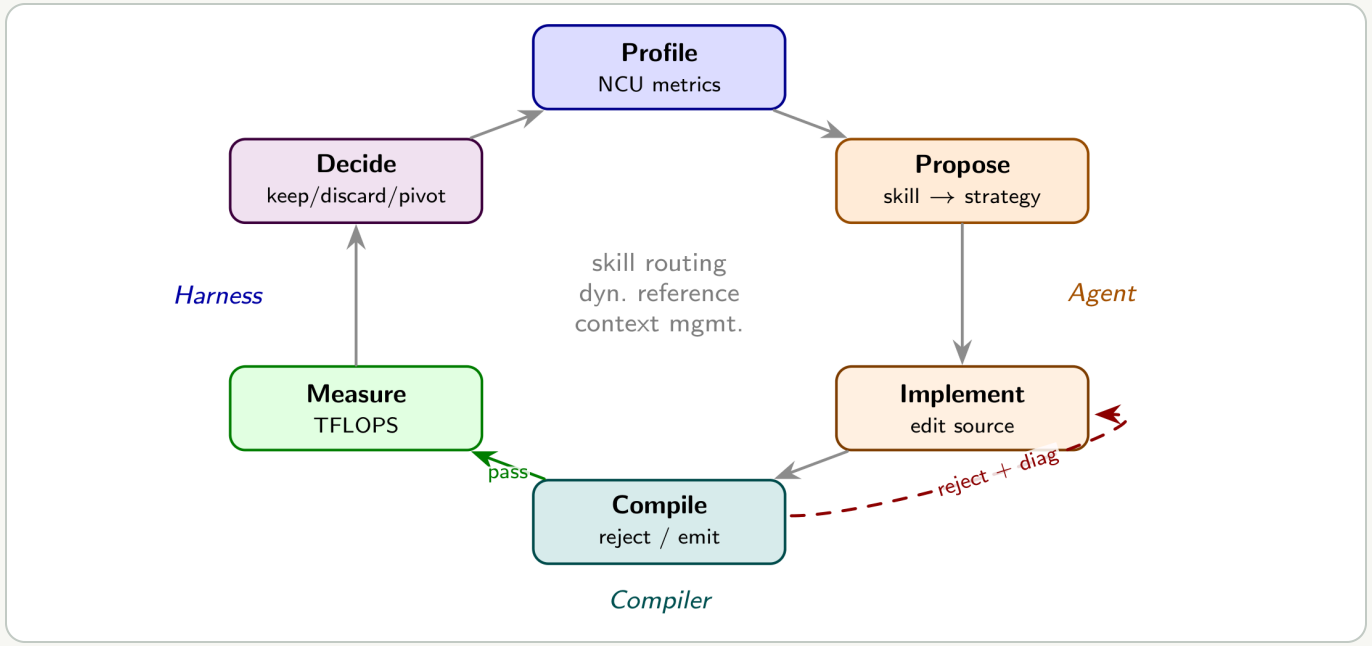
- **480 configurations → 102 viable**

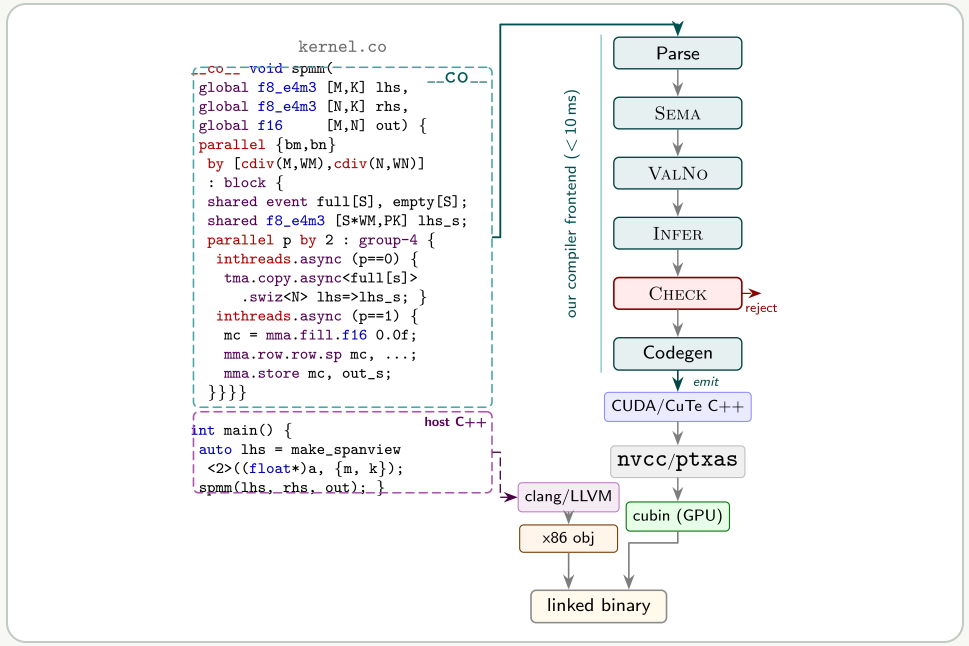
  79% are pruned before GPU execution, mostly for register and shared-memory limits.

- **The harness measures viable candidates**

  Rejected candidates return with a repair hint; passing candidates proceed to correctness checks and timing.

Close the autonomous loop





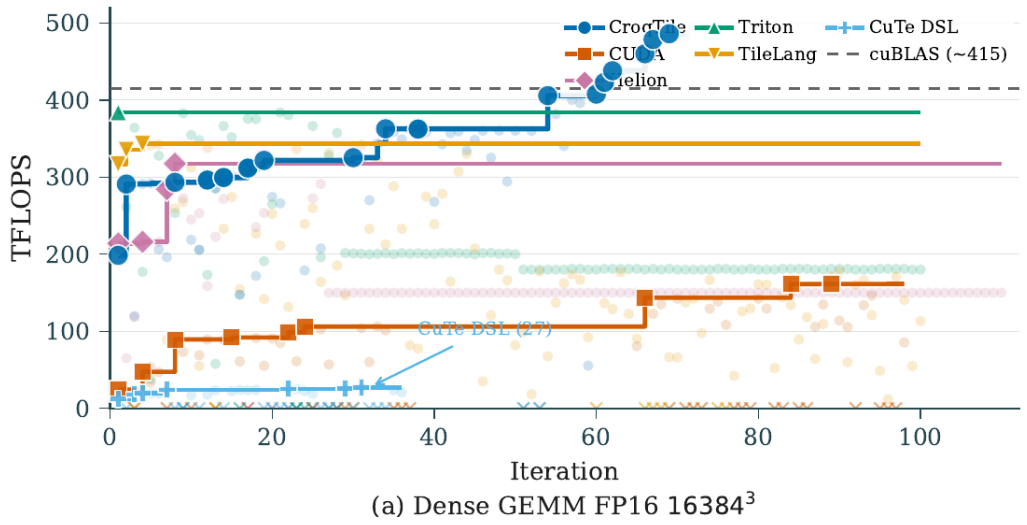
- **Syntax–context co-design**
  Compact source and local edits keep prior attempts and measurements available to the agent.
- **Compiler–harness co-design**
  Structured rejection feeds the next repair; valid candidates proceed to verification and measurement.

1.23× vendor libraries Geometric-mean speedup across evaluated workloads with vendor references.

Tuning beyond cuBLAS

Dense FP16 GEMM

M = N = K = 16,384

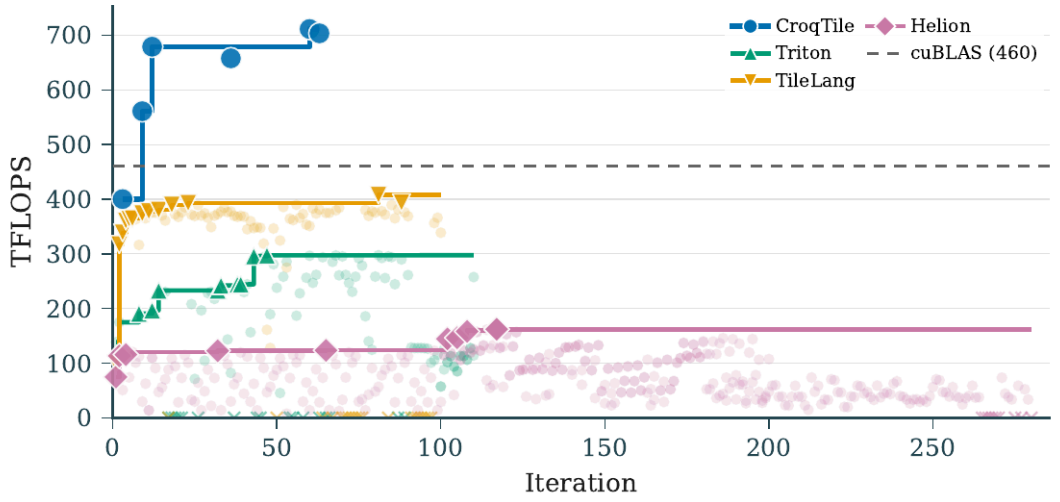


(a) Dense GEMM FP16 16384^3

Improves steadily, reaches cuBLAS, then exceeds it.

Block-scale E4M3 GEMM

M = N = K = 8,192



(b) Block-scale GEMM E4M3 8192^3

Crosses the vendor baseline early and continues improving.

- **Read the curves**
  Solid lines: running-best TFLOPS; dashed line: cuBLAS.

- **Shared evaluation protocol**
  H800 PCIe · Claude Sonnet 4.6 · same agent and per-shape time cap.

Thank you



CroqTile is open source

Scan · CroqTile repo

ACCEPTED WORKS USING CROQTILE

SC'26 · Structured Sparse GEMM on Hopper via Analytical Modeling

SC'26 · Do We Need Tensor Cores for Stencil Computations? **Best Paper Nominee**

ATC'26 · An AI-Native GPU Kernel Programming Language for Autonomous Tuning

ATC'26 · SptcAttn: Long-Context Attention with Diagonal-Stripes Sparse Tensor Core

CGO'27 · Signal-Aware Liveness Analysis for Shared Memory in Warp-Specialized Kernels